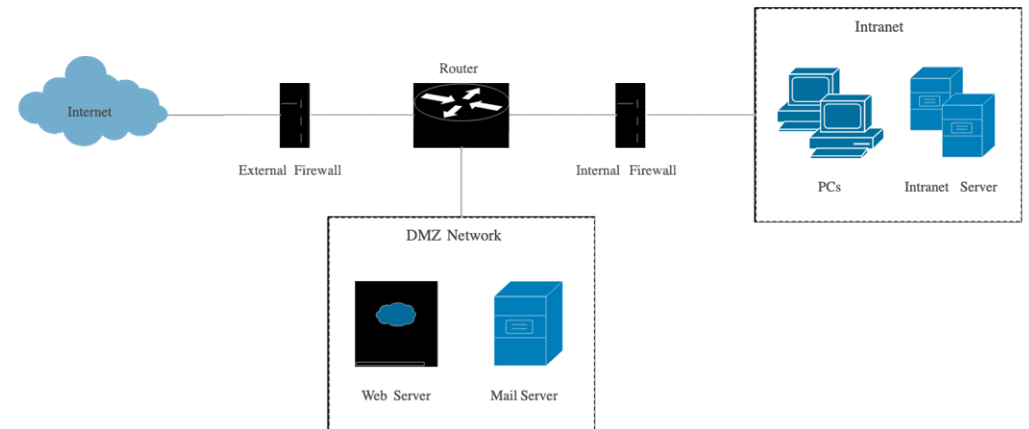
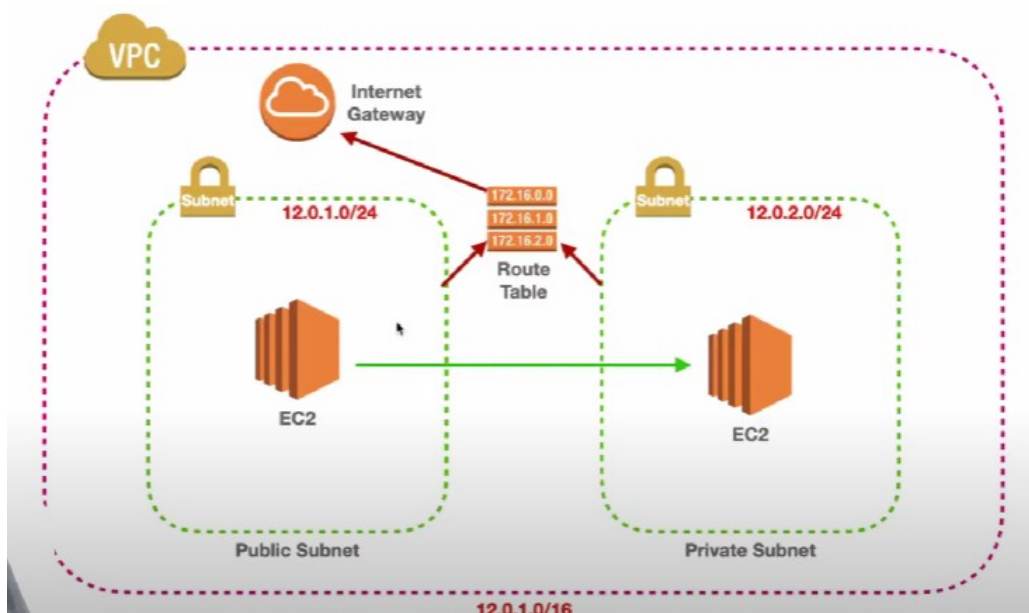


AWS VPC

Topics

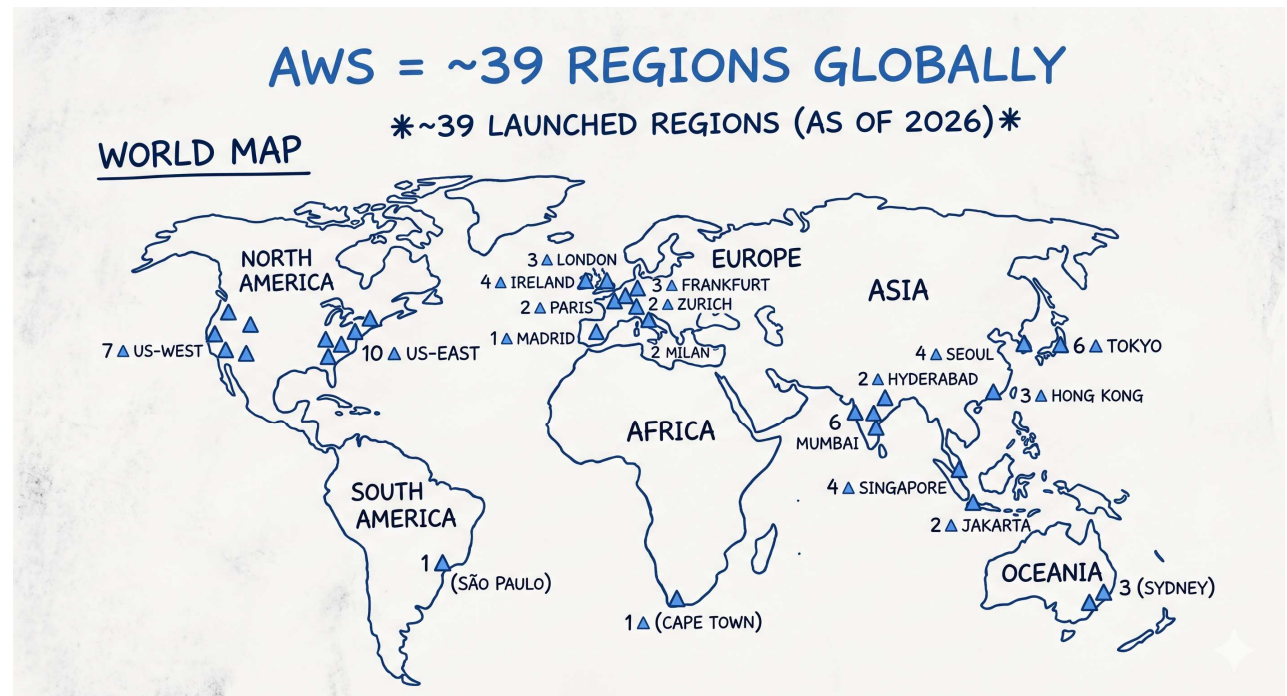
- AWS Services scope
- VPC
- VPC Firewall
- Connect your VPC
- VPC with other services

Traditional on-premises network vs Virtual Network in the Cloud



What is a Region?

- A **Region** is a **geographical area** where AWS has data centers
- **Example**
 - AWS Mumbai Region
 - AWS Singapore Region



What is an Availability Zone (AZ)?

- An **Availability Zone (AZ)** is a **separate data center inside a region**. Each AZ is:
 - Independent (power, cooling, network)
 - Connected with low latency
 - High Availability
 - Fault Isolation
- Region (City)
 - ├── AZ1 (Building A)
 - ├── AZ2 (Building B)
 - └── AZ3 (Building C)

AWS Service Scope

Global Services

Services that are **not tied to any specific region**

One configuration → works worldwide

Examples

- AWS Identity and Access Management
- Amazon CloudFront
- Amazon Route 53

Regional Services

Services that are deployed **within a specific region**

Exists in one region only

Examples

- Amazon VPC
- Amazon RDS
- Amazon S3

Availability Zone (AZ) Services

Services that run in a **specific AZ**

Tied to a physical data center

• Examples

- Amazon EC2
- Subnets
- EBS volumes

AWS Service Scope Hierarchy



GLOBAL SERVICES

 IAM

 Route 53

 CloudFront



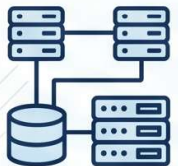
REGIONAL SERVICES

REGION

 VPC

AZ
 S3

 RDS



AZ SERVICES

AZ



 EC2

AZ



 Subnets

AZ



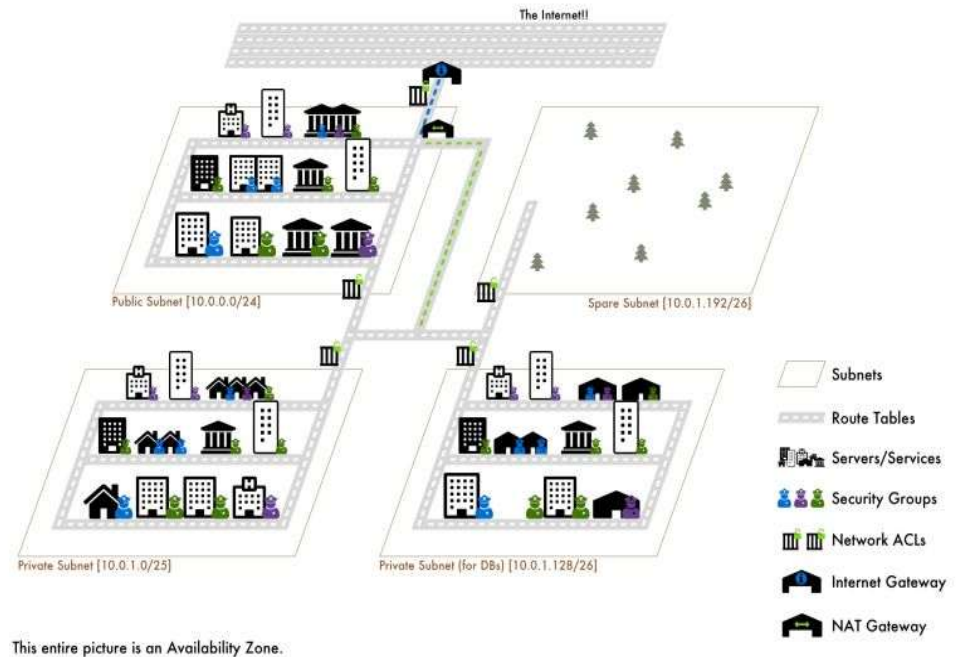
 EBS

Virtual Private Cloud (VPC)

- VPC is a logically isolated network in AWS, its similar to your on-perm data center network

“VPC = your private colony in a city”

- City = AWS
- Colony = your VPC
- You decide:
 - Roads (routing)
 - Houses (servers)
 - Security (firewalls)



VPC Service Scope

AWS services inside VPC and outside VPC

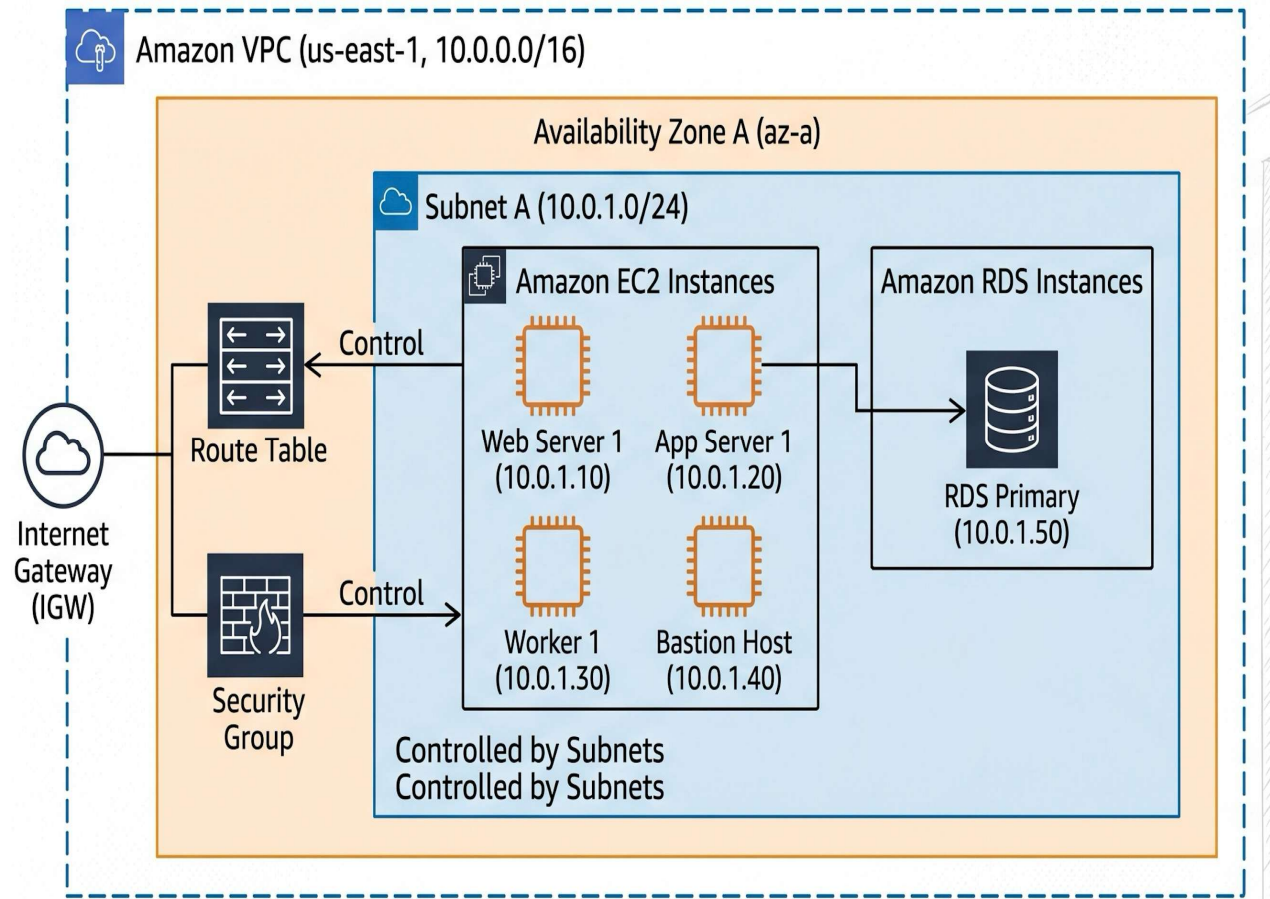
Some AWS services live inside your VPC (you control networking), others live outside (AWS manages them)

Services Inside VPC

Services that run **inside your private network (VPC)** and get IP addresses from it thus Controlled by Subnets, Route tables and Security Groups

- **Examples**

- Amazon EC2
- Amazon RDS
- Application Load Balancer

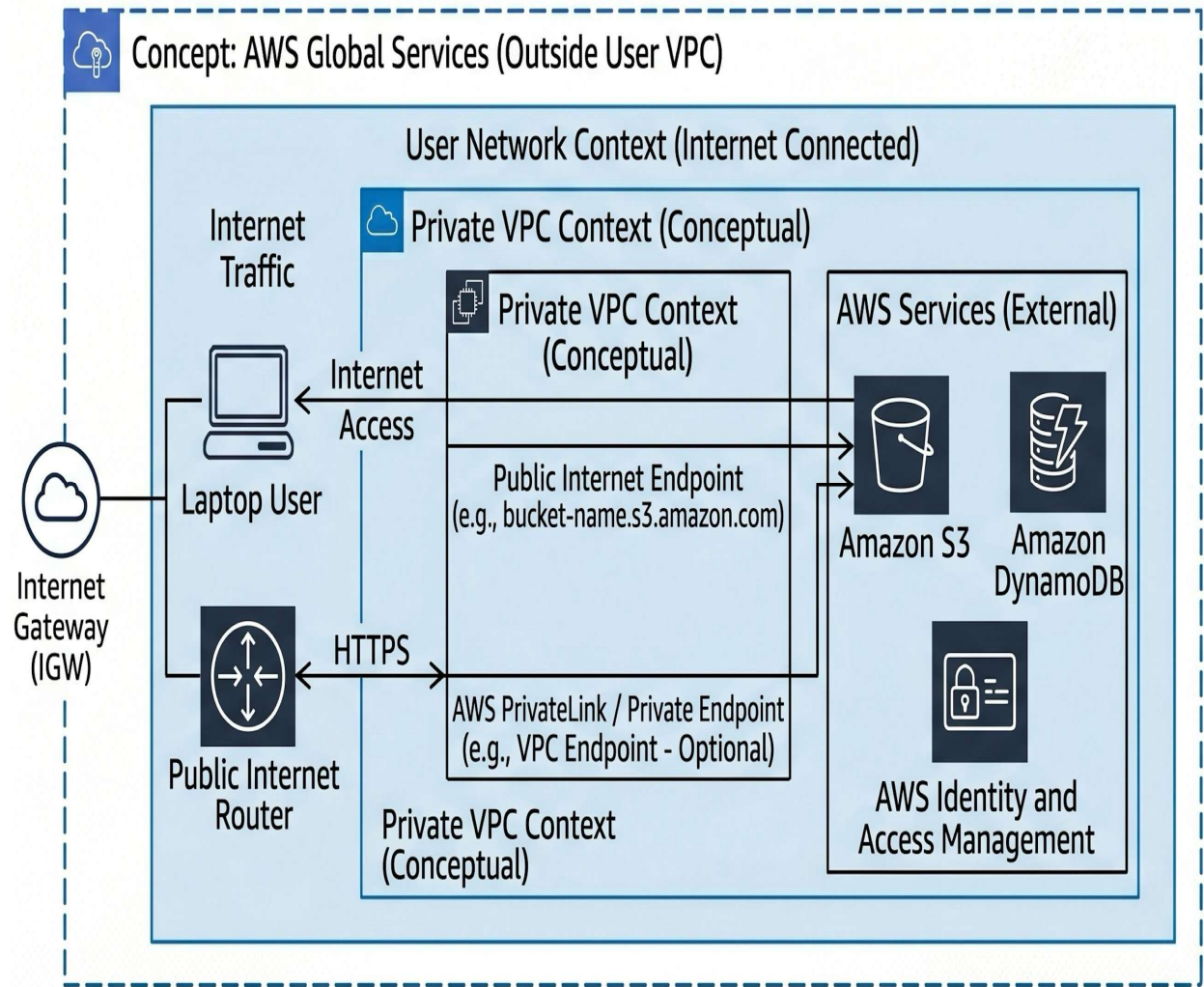


Services Outside VPC

Services that are **not deployed inside your VPC** (fully managed by AWS), thus No VPC IP address and accessed via Internet or Private endpoints

Examples

- Amazon S3
- Amazon DynamoDB
- AWS Identity and Access Management



VPC Building Blocks

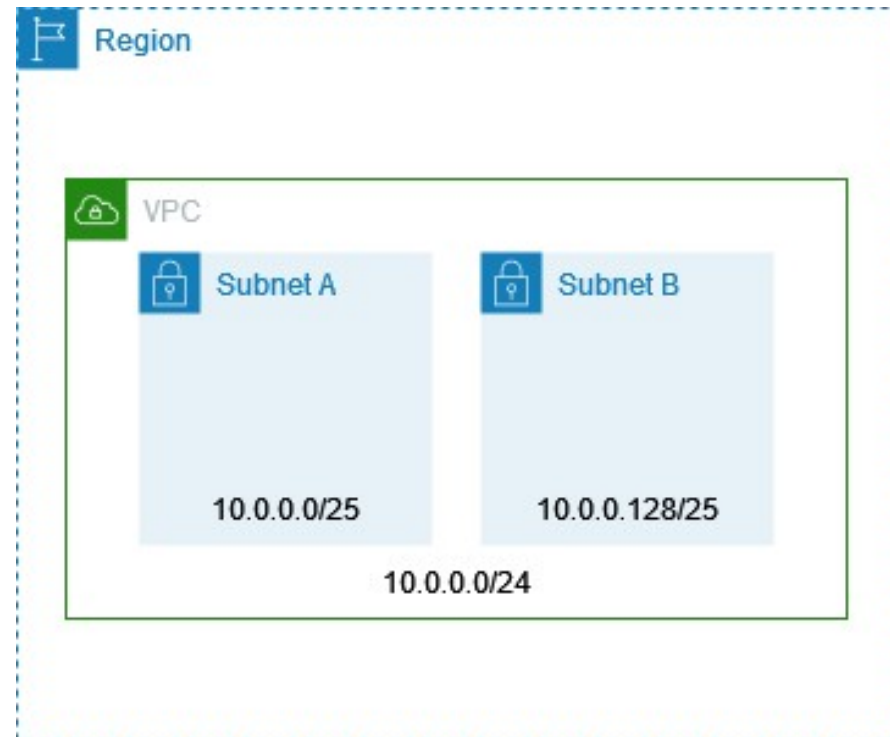
- Subnets (Segmentation)
- Route Tables (Traffic Control)
- Security Groups (Instance Firewall)
- Network ACL (Subnet Firewall)
- Internet Gateway (IGW)
- NAT Gateway
- Elastic IP
- VPC Endpoints

VPC CIDR

CIDR - Ipv4 VPC

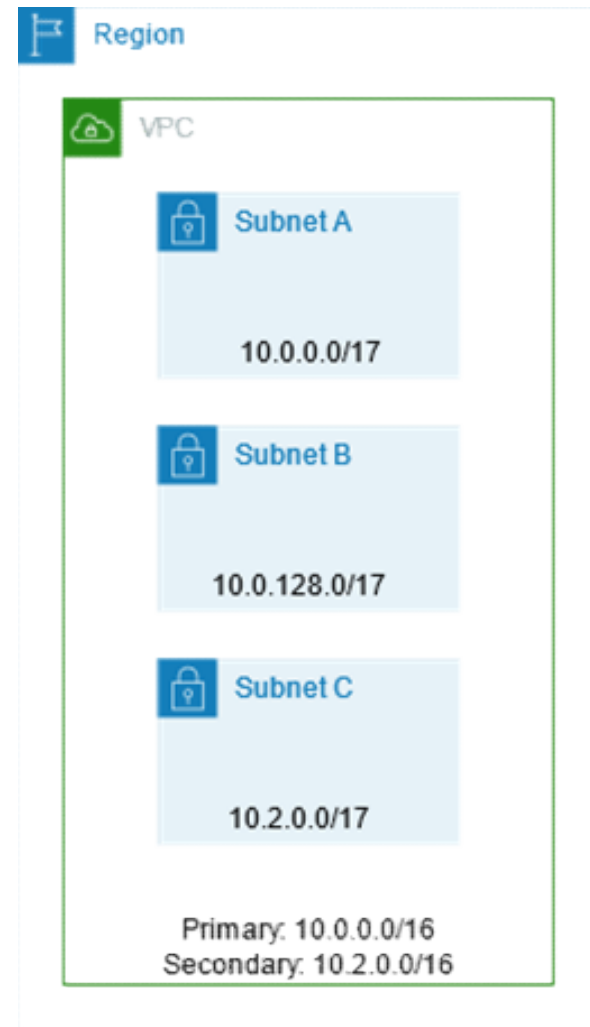
- When you create a VPC, you must specify an IPv4 CIDR block for the VPC
- Allowed block size is between a /16 netmask (65,536 IP addresses) and /28 netmask (16 IP addresses).
- Recommend / Allowed CIDR block from the private IPv4 address.

RFC 1918 range	Example CIDR
10.0.0.0 - 10.255.255.255 (10/8 prefix)	10.0.0.0/16
172.16.0.0 - 172.31.255.255 (172.16/12 prefix)	172.31.0.0/16
192.168.0.0 - 192.168.255.255 (192.168/16 prefix)	192.168.0.0/20



CIDR – IPv4 VPC

- Can't specify the following CIDR blocks for your VPCs:
 - 0.0.0.0/8
 - 127.0.0.0/8 (internal host loopback address range)
 - 169.254.0.0/16 ([link-local address range](#))
 - 224.0.0.0/4 (multicast address range)
- You can associate secondary IPv4 CIDR blocks with your VPC
- You can add up to 5 CIDR block, 1 primary mandatory, 4 secondary optional.



CIDR – IPv6 VPC

- You can associate **1 IPv6 CIDR block at VPC creation and add up to 5 IPv6 CIDR blocks** later.
- Supported range: **/44 to /60**. Allocation in **increments of /4**
- IPv6 CIDR is **assigned by AWS (Amazon pool), you cannot choose it.**
- Subnets get IPv6 from VPC CIDR, Typical subnet size: **/64**

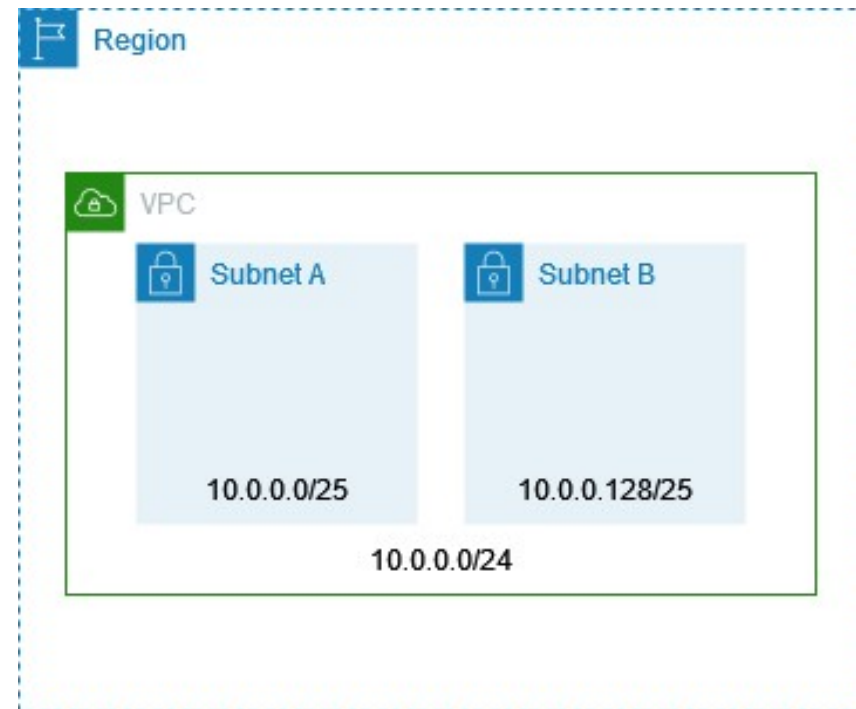
Example:

- VPC: 2001:db8:1234:1a00::/56
 - Subnet: 2001:db8:1234:1a00::/64
-
- You can configure Both together(**Dual-stack VPC**) IPv4 CIDR (mandatory) , IPv6 CIDR (optional).

VPC Subnets

Subnets

- **Subnet** can be same as CIDR block for the VPC or is a **sub-section of it**.
- All subnets must be within the VPC CIDR range. It cannot overlap.
- Each subnet resides in **one Availability Zone (AZ)**.
- Example:
 - VPC: 10.0.0.0/24 → 256 Ips
 - Split into two subnets:
 - Subnet 1: 10.0.0.0/25 → 128 IPs (0–127)
 - Subnet 2: 10.0.0.128/25 → 128 IPs (128–255)



Subnets sizing for IPv4

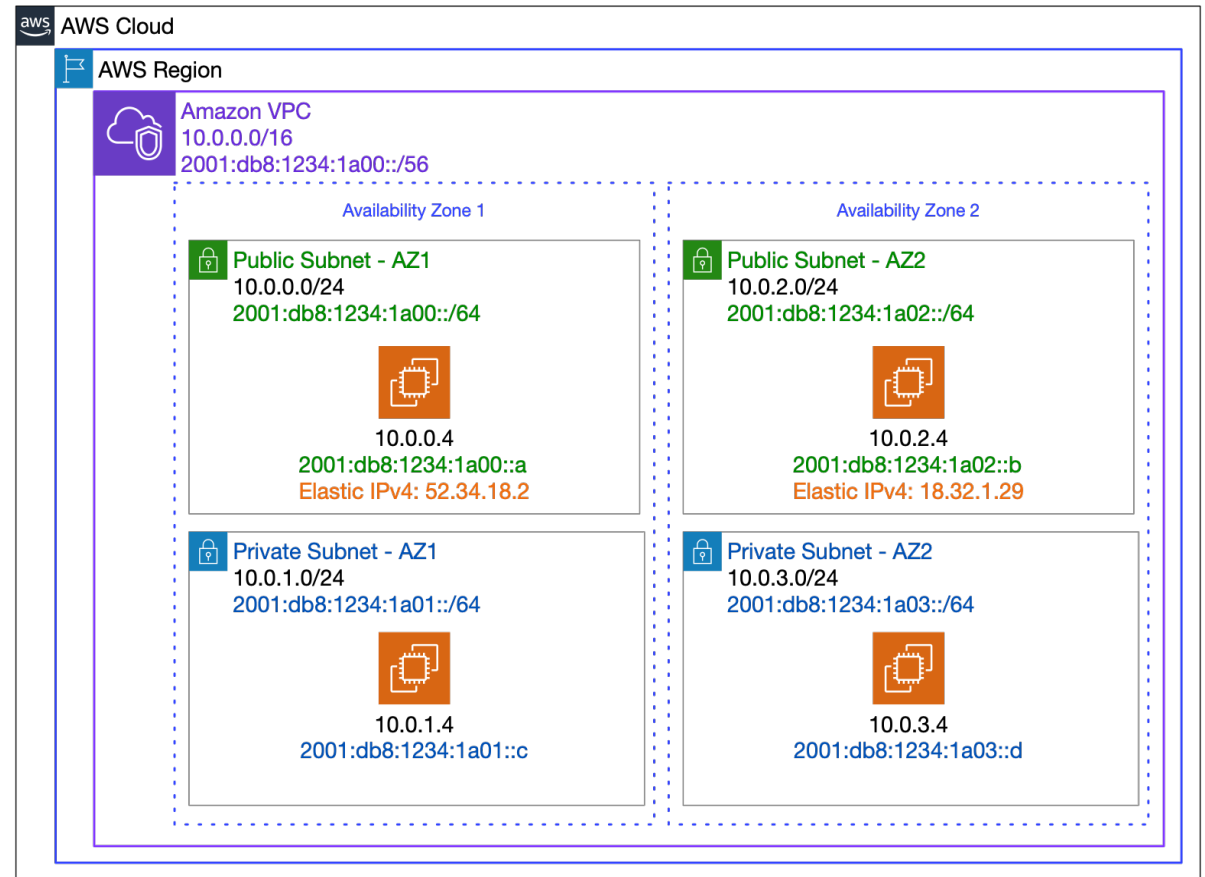
- AWS reserves 5 Ips (first 4 and last 1) in each subnet
- These 5 Ips are not available for use and cannot be assigned.
- If CIDR 10.0.1.0/24 is Subnet Range
 - 10.0.1.0 : Network address
 - 10.0.1.1: Reserved for VPC router
 - 10.0.1.2: Reserved for DNS
 - 10.0.1.3: Reserved for future use
 - 10.0.1.255: Reserved for broadcast . AWS doesn't support broadcast in VPC.
- If you need 29 Ip for your use in VPC, you can't choose a subnet of size/27(32 IP)

Subnets sizing for IPv6

- Typical subnet size: **/64**
- First **4 IPv6 addresses** → reserved
- Last **1 IPv6 address** → reserved

VPC having both IPv4 and IPv6 subnets

- VPC supports dual stack mode
- Resources inside VPC can communicate over IPv4 and IPv6 or both
- IPv4 support cannot be disabled for VPC



Types of Subnets

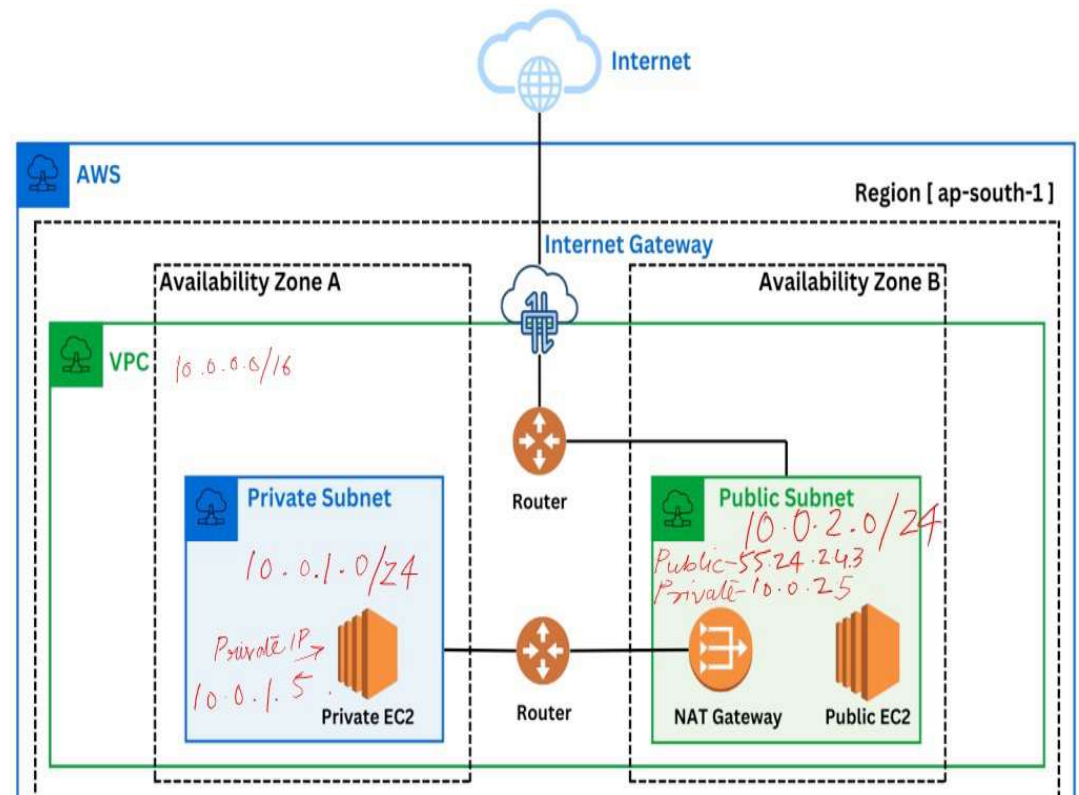
Public Subnet:

- Has route to Internet Gateway
- Used for: Load balancers, bastion hosts, NAT, Web servers

Private Subnet

- No direct internet access
- Used for: App servers, databases

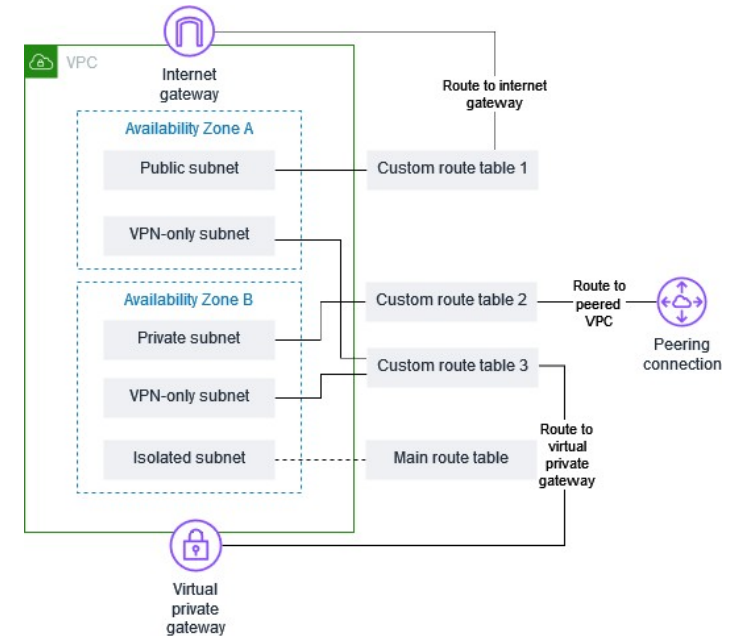
There is NO “public subnet” option in AWS.
A subnet becomes **public or private based on its routing configuration**, not a checkbox.



VPC Route Table

Route Table

- **Contains** set of rules that decide **where network traffic should go**.
- **Route Table is** associated with subnets.
- 2 types of Route Table, Main and Custom.
- Each route table contains default immutable local route for VPC



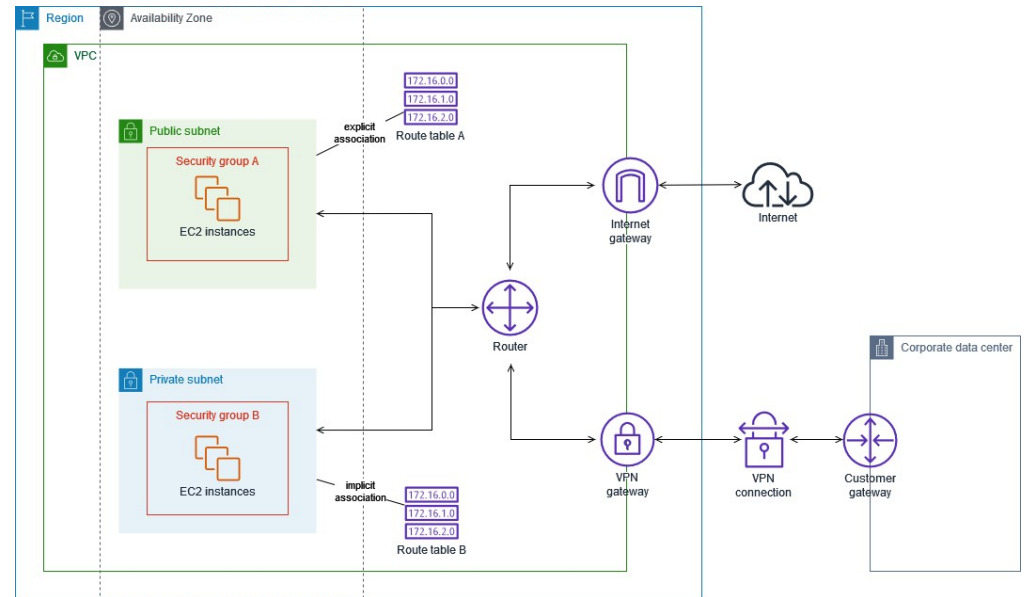
Destination	Target
10.0.0.0/16	Local
2001:db8:1234:1a00::/56	Local
172.31.0.0/16	pcx-11223344556677889
0.0.0.0/0	igw-12345678901234567
::/0	eigw-aabbccdde1122334

Types of Route Table

Feature	Main Route Table	Custom Route Table
Creation	Created automatically with Amazon VPC	Created manually by you
Default Behavior	All subnets use it by default	No subnet uses it unless you associate
Purpose	Basic / fallback routing	Specific routing design (public/private)
Association	Implicit (default)	Explicit (manual)
Flexibility	Limited (not recommended for complex setups)	High (full control)
Typical Routes	Only local route initially	IGW, NAT, peering, etc.
Usage	Small/simple setups	Production architectures

Main and Custom Route Table

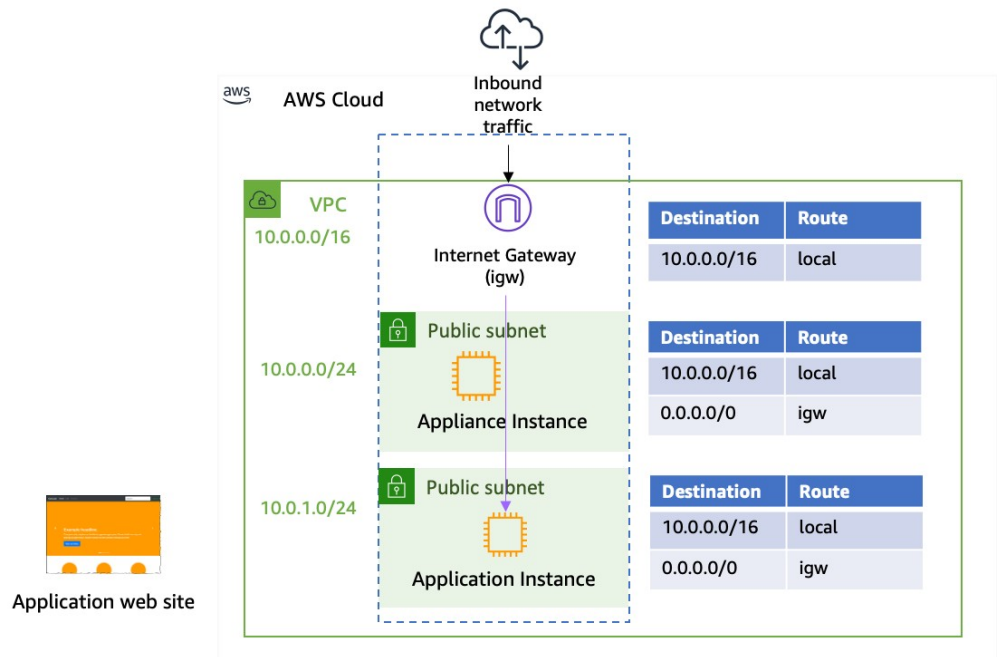
- Route table A is a custom route table that is explicitly associated with the public subnet.
- Route table B is the main route table. It is implicitly associated with the private subnet.



Routing to Internet Gateway (Public Subnet):

- Create & attach Internet Gateway
- Add route in subnet route table:
0.0.0.0/0 → IGW (IPv4)
::/0 → IGW (IPv6)

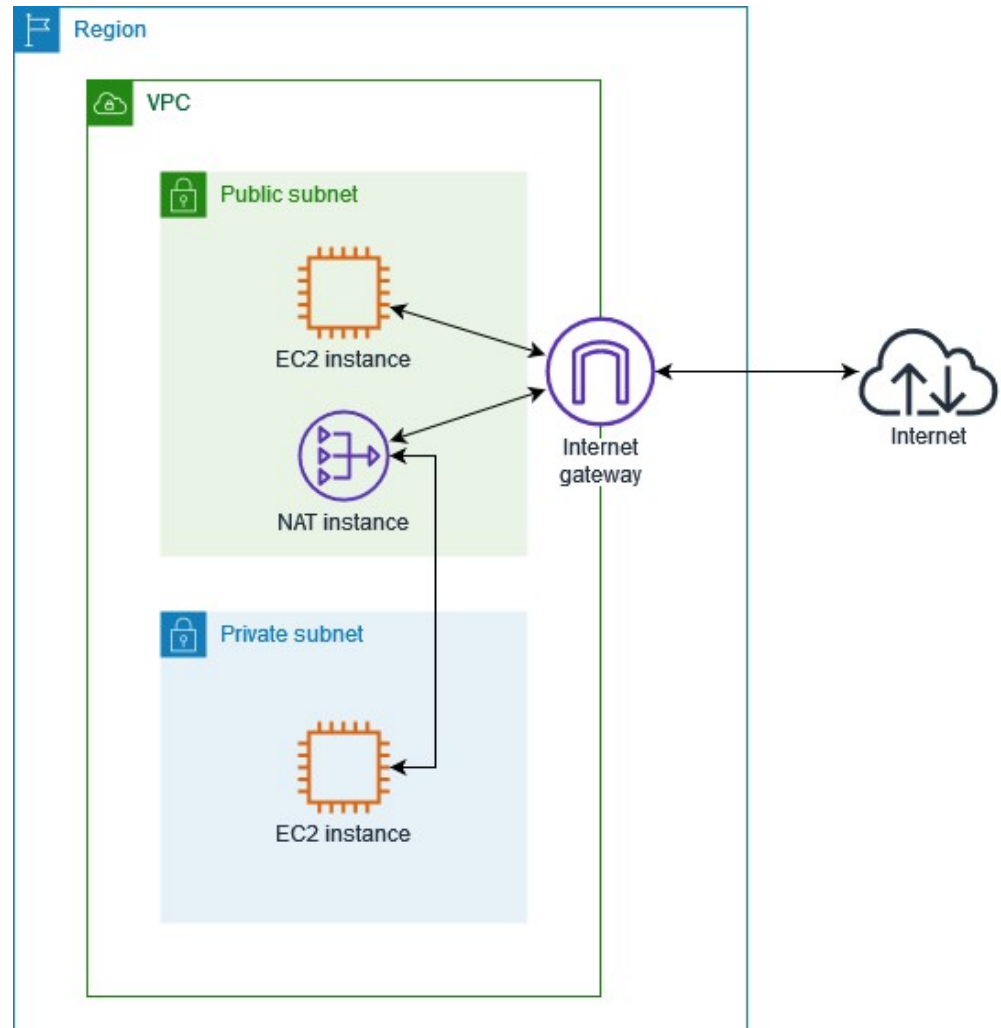
This will makes subnet public and
Enables **direct internet access**



Routing to NAT Device (Private Subnet)

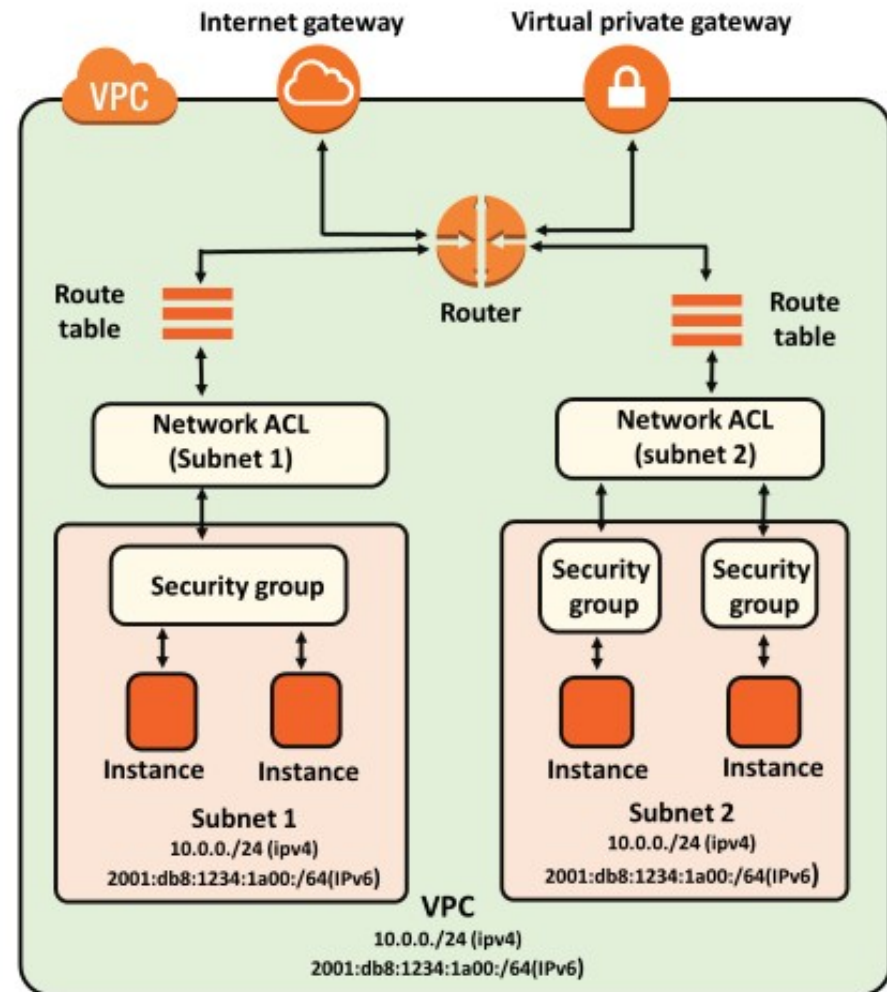
- Create NAT Gateway or NAT instance
- Place it in public subnet
- Add route
0.0.0.0/0 → NAT Gateway

This will enable outbound internet for private instances



VPC Firewalls

- Security Groups
- Network Access Control List (NACL)



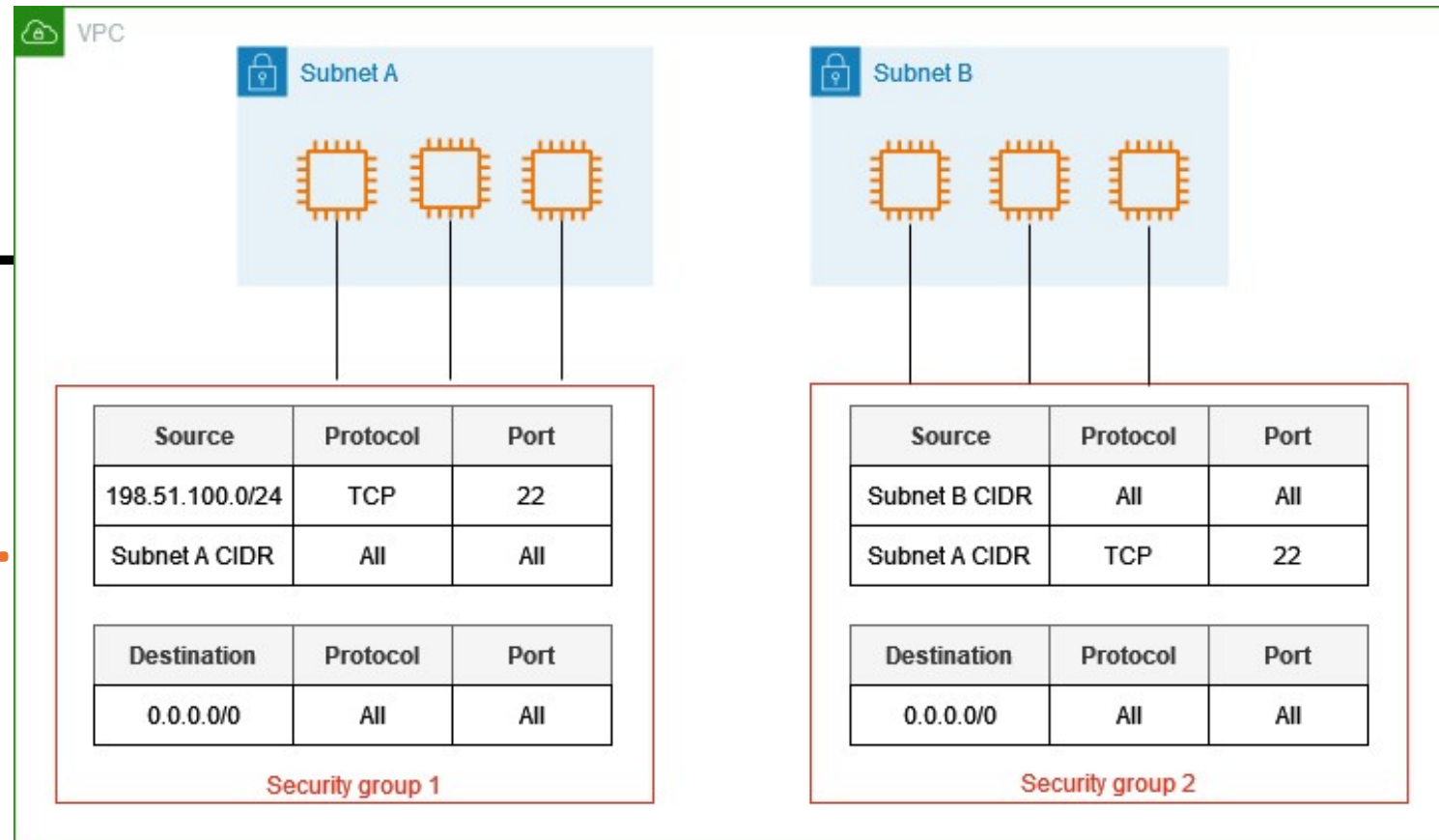
VPC Security Group

Security Groups

- Security Groups are the fundamental of network security in AWS
- They control how traffic is allowed into or out of our EC2 Machines(**Instance level**).
- It is the most fundamental skill to learn to troubleshoot networking issues
- In this session, we'll learn how to use them to **allow**, **inbound** and **outbound** ports



Security Groups - Diagram



Security Group Example

Setup

- VPC with **2 subnets (A & B)**
- Each subnet has its own security group:
 - Subnet A → Security Group 1
 - Subnet B → Security Group 2

Security Group 1 (Subnet A)

- Allows **SSH (port 22)** from trusted IP range (e.g., office network)
- Allows **all traffic within subnet A** (instance-to-instance communication)

Security Group 2 (Subnet B)

- Allows **all traffic within subnet B**
- Allows **SSH access from subnet A**

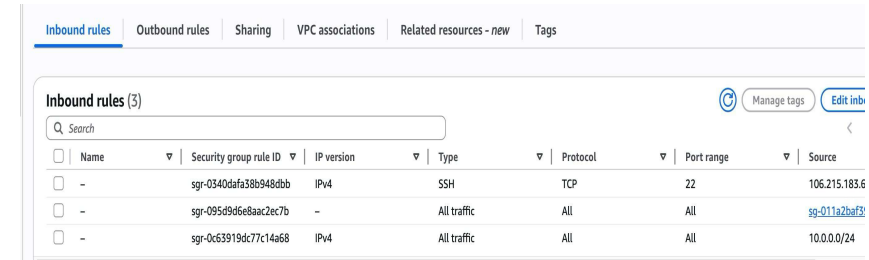
Outbound Rules

- Both security groups:
 - Allow **all outbound traffic (default rule)**

Key Insights

- Security Groups are **instance-level firewalls**
- They are **stateful** (response traffic allowed automatically)
- Can reference **other security groups** (A → B communication)

Security Groups – Key Concepts

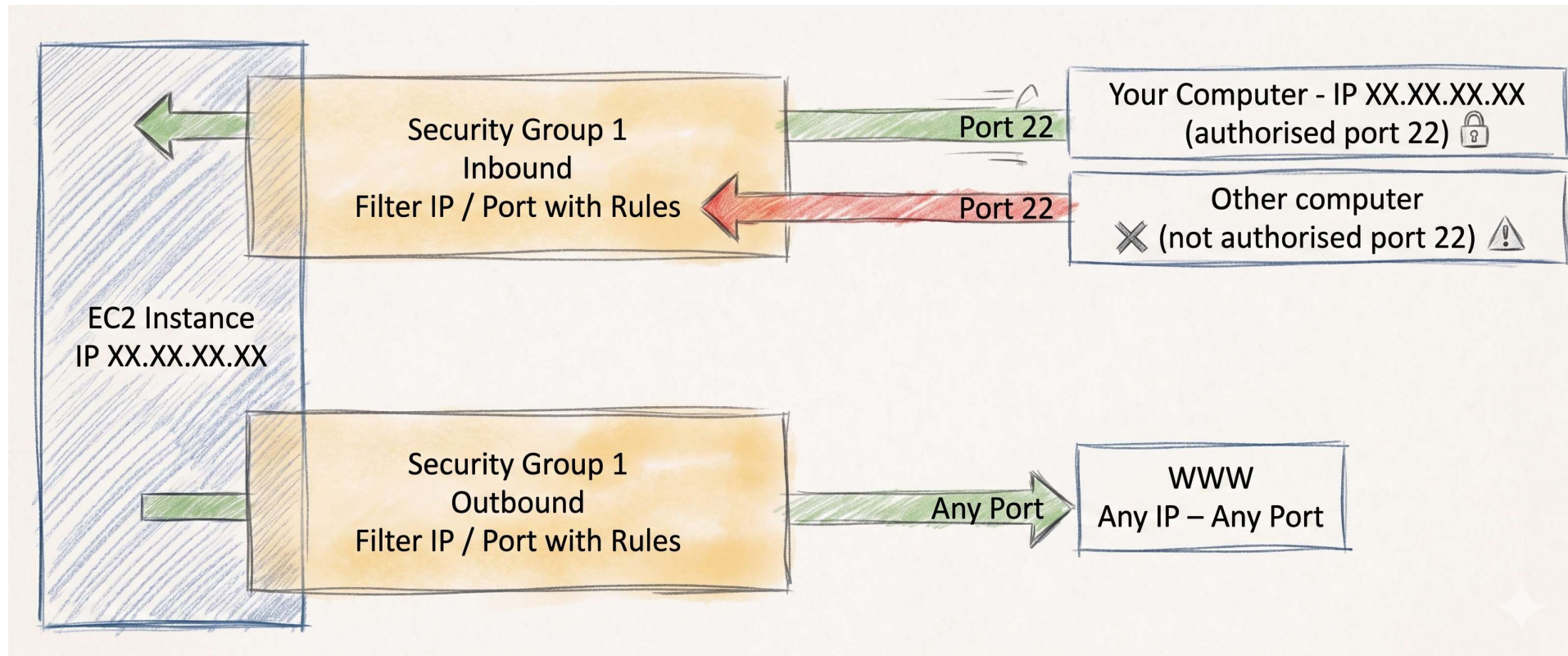


The screenshot shows the AWS Management Console interface for a Security Group's Inbound rules. The top navigation bar includes tabs for Inbound rules, Outbound rules, Sharing, VPC associations, Related resources - new, and Tags. The 'Inbound rules' tab is active, showing a list of 3 rules. A search bar is present above the table. The table columns are Name, Security group rule ID, IP version, Type, Protocol, Port range, and Source. The first rule is for SSH (TCP) on port 22 from source 106.215.183.6. The second rule is for All traffic (All) on all ports from source sg-011a2ba7. The third rule is for All traffic (All) on all ports from source 10.0.0.0/24.

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
-	sgr-0340dafa38b948dbb	IPv4	SSH	TCP	22	106.215.183.6
-	sgr-095d9d6e8aac2ec7b	-	All traffic	All	All	sg-011a2ba7
-	sgr-0c63919dc77c14a68	IPv4	All traffic	All	All	10.0.0.0/24

- Can be **attached to multiple instances**
- Scoped to a **specific VPC and Region** in Amazon VPC
- Operate **outside the instance** → blocked traffic never reaches the instance
- Have **two rule sets: Inbound and Outbound**
- If observing **Timeout issue** → likely **Security Group blocking traffic**
- If observing **Connection refused** → application issue or service not running
- **Default Behavior** Inbound traffic → **✗ Blocked by default**
- **Default Behavior of** Outbound traffic → **✓ Allowed by default**

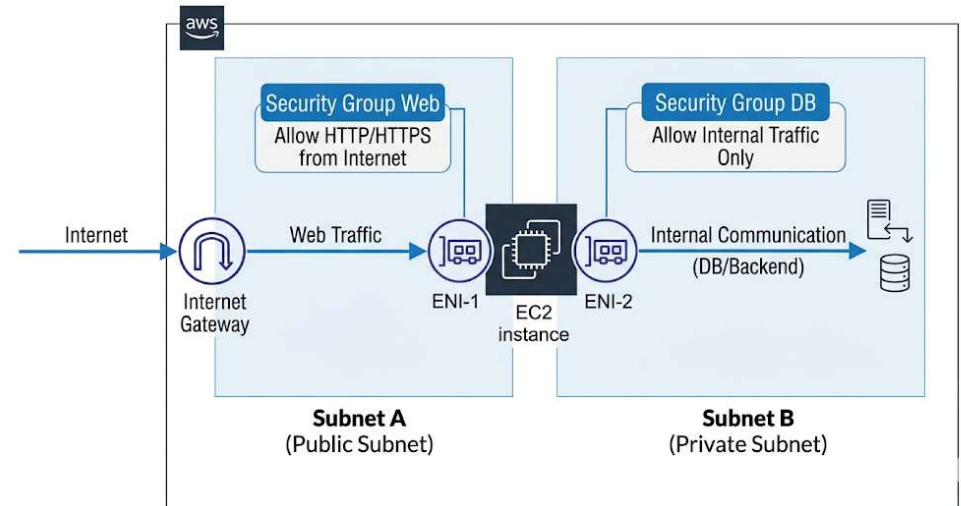
Security Groups - Diagram



Security Groups

– Key Insights

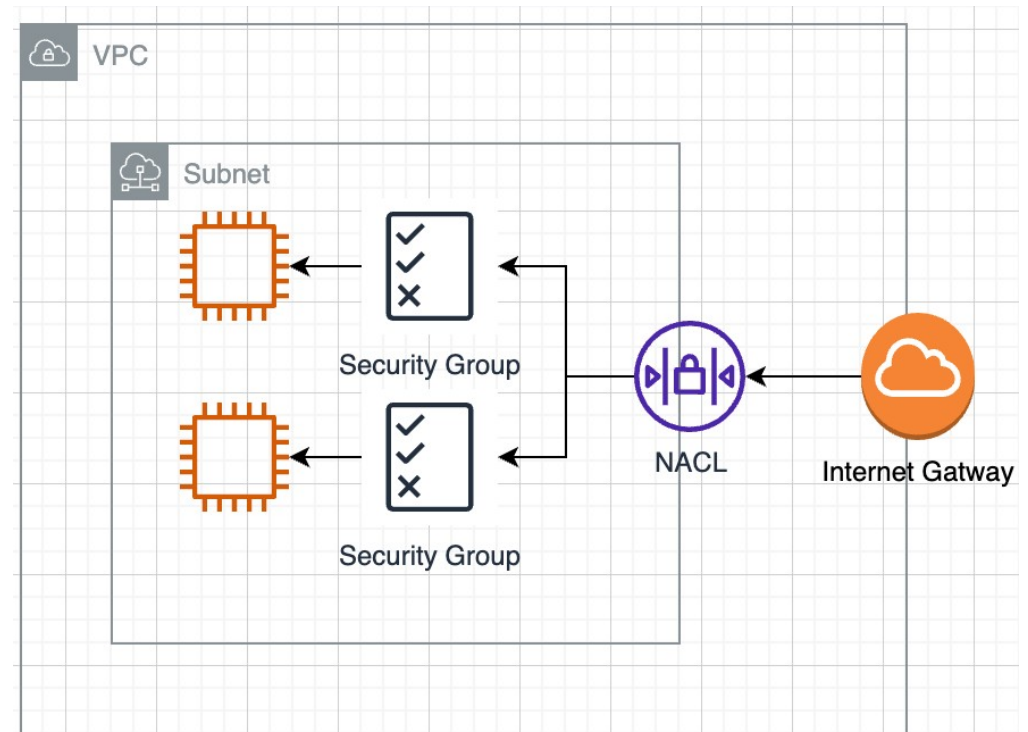
- Security Groups are **instance-level firewalls** in Amazon VPC
- Applied to **instances/ENIs**, not subnets.
- Traffic between instances is allowed **only if both SGs permit (inbound + outbound)**.
- Security Groups are **stateful** (response traffic auto-allowed).
- You can **reference another Security Group** in source instead of Ips.
- No deny rules → only **allow-based security model**.
- Can be attached to **multiple instances**



VPC Network Access Control List (NACL)

Network Access Control List (NACL)

- In Core **Network ACL (NACL)** is a **subnet-level firewall** that controls traffic **in and out of a subnet**
- It Works as an **additional security layer** on top of Security Groups
- Associated with **subnets, not individual resources**
- Each subnet can have **only one NACL at a time**
- Re-associating subnet → removes previous NACL
- Its Supports both: ✓ Allow rules & ✗ Deny rules
- Useful for **blocking specific IPs (blacklisting)**
- NACL is **stateless** thus Inbound and outbound rules must be **defined separately**
- Response traffic is **not automatically allowed**



Network Access Control List (NACL) Continue..

- Rules are evaluated in **ascending order (rule number)**
- First rule match wins once its matched further evaluating of rule is stopped.
- Specific rules should have **higher priority (lower number)**
- Default VPC NACL: Allows **all inbound & outbound traffic** & Cannot be deleted
- Custom NACL: Denies **all traffic by default** & Must explicitly allow traffic
- Subnets without custom NACL → use **default NACL**
- NACL can be associated with **multiple subnets**
- **Traffic Evaluation Order for Inbound Traffic:** NACL rules (ordered) → Security Group rules → If no allow → **✗** blocked
- **Traffic Evaluation Order for Outbound Traffic:** Security Group rules → NACL rules

Summary

Inbound Rules

Outbound Rules

Subnet Associations

Tags

Allows outbound traffic. Because network ACLs are stateless, you must create inbound and outbound rules.

Edit

Rule #	Type	Protocol	Port Range	Destination	Allow / Deny
1	All ICMP	ICMP (1)	ALL	0.0.0.0/0	ALLOW
100	Custom TCP Rule	TCP (6)	1024-65535	0.0.0.0/0	ALLOW
200	HTTP (80)	TCP (6)	80	0.0.0.0/0	ALLOW
300	HTTPS (443)	TCP (6)	443	0.0.0.0/0	ALLOW
400	SSH (22)	TCP (6)	22	10.0.0.0/16	ALLOW
500	MySQL/Aurora (3306)	TCP (6)	3306	10.0.0.0/16	ALLOW
*	ALL Traffic	ALL	ALL	0.0.0.0/0	DENY

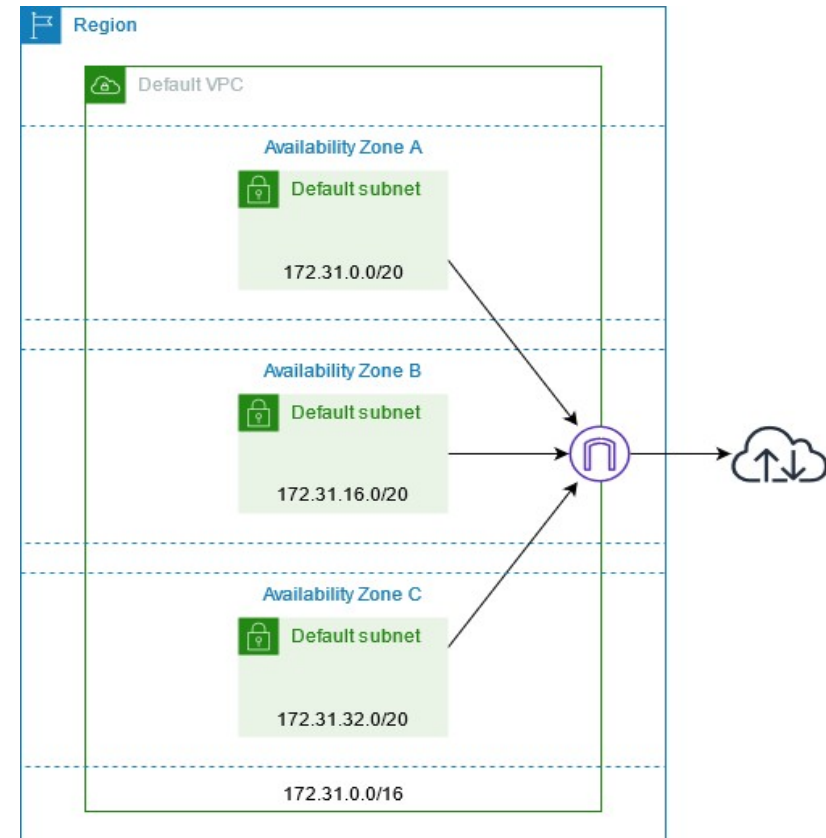
Network ACLs vs Security Groups

Feature	Security Groups	Network ACL (NACL)
Scope	Instance / ENI level	Subnet level
Type	Stateful	Stateless
Rules	Allow only	Allow & Deny
Rule Evaluation	All rules evaluated (no order)	Evaluated in order (rule number)
Default Behavior	Deny inbound, Allow outbound	Default NACL: Allow all
Response Traffic	Automatically allowed	Must be explicitly allowed
Association	Multiple SGs per instance	One NACL per subnet
Flexibility	Fine-grained	Coarse-grained
Use Case	Whitelisting traffic	Blacklisting / blocking traffic

Default VPC

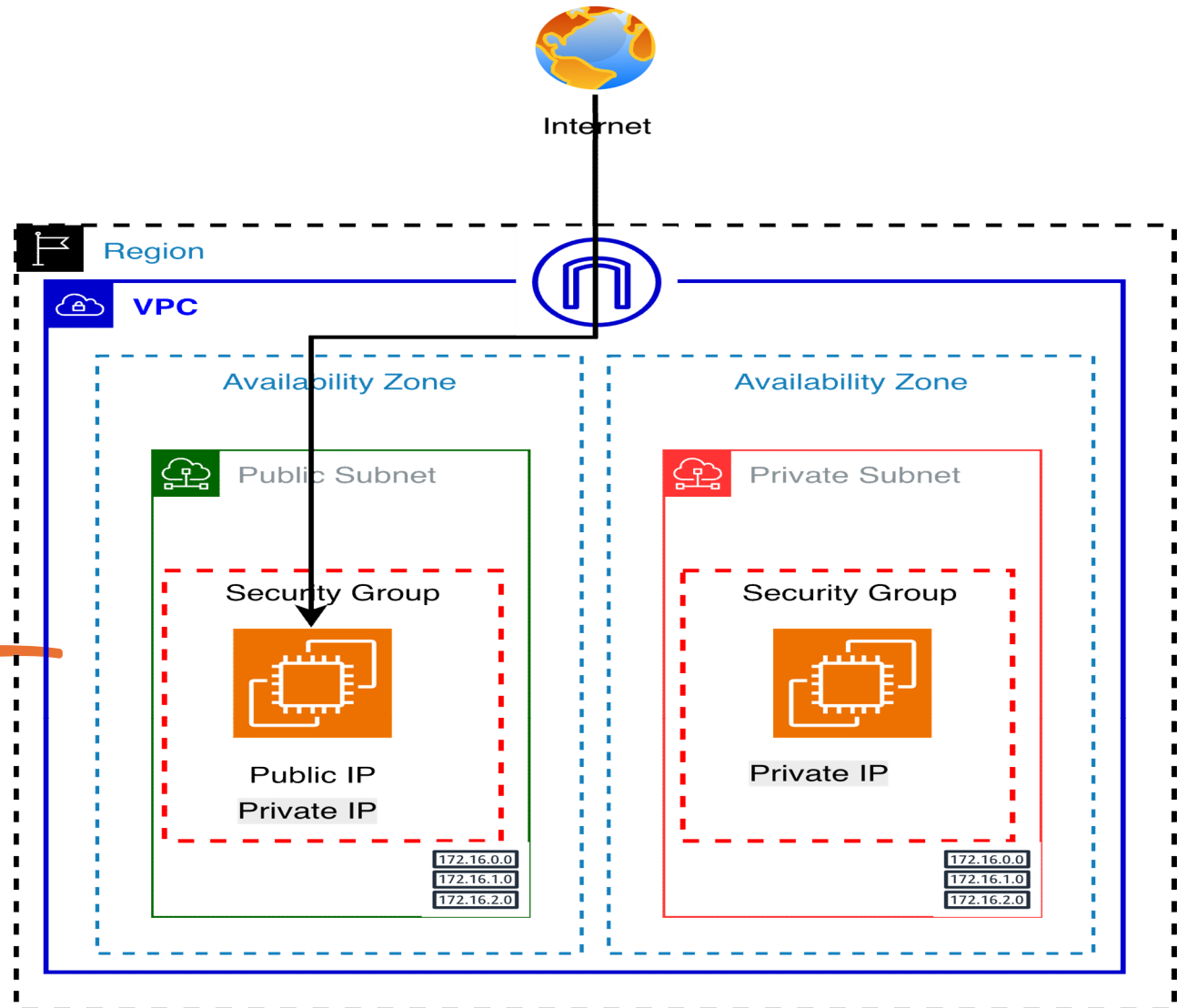
Default VPC

- Ready-to-Use Network, Automatically created in every AWS Region upon account setup.
- Designed to let users launch EC2 instances immediately without manual networking.
- Uses a fixed CIDR block (172.31.0.0/16) to ensure a consistent experience for beginners.
- Comes with an Internet Gateway (IGW) pre-attached and ready for public traffic.
- One public subnet automatically created in each Availability Zone (AZ).
- Includes a pre-configured Main Route Table and an Internet Gateway (IGW).
- Includes a Default Security Group and Network ACL (NACL) allowing all outbound traffic.
- DNS Resolution and DNS Hostnames are enabled by default for easy resource naming.



Destination	Target
172.31.0.0/16	local
0.0.0.0/0	<i>internet_gateway_id</i>

AWS VPC Hands-on Lab



VPC Hands-on Part 1

- Remove the default VPC (we will create a custom VPC from scratch).
- Create a new VPC
 - a. Navigate to the VPC service → **Your VPCs** → **Create VPC**
 - b. Configure the following details:
 - Name: **TestVPC**
 - IPv4 CIDR Block: **10.100.0.0/16**
 - c. Click **Create VPC**
- Create an Internet Gateway
 - a. Go to **Internet Gateways** → **Create Internet Gateway**
 - b. Provide the name **TestVPC-IGW** and create the gateway.
- Attach the Internet Gateway to the VPC
 - a. Select the created Internet Gateway
 - b. Choose **Actions** → **Attach to VPC**
 - c. Select **TestVPC** and attach it.
- Create a Public Subnet
 - a. Navigate to **Subnets** → **Create Subnet**
 - b. Enter the following details:
 - Name: **TestVPC-Public**
 - VPC: **TestVPC**
 - Availability Zone: **ap-south-1a**
 - CIDR Block: **10.100.0.0/24**
 - c. Create the subnet.
 - d. After creation, select the subnet → **Actions** → **Modify Auto-Assign Public IP**
 - e. Enable **Auto-Assign Public IPv4 Address** and save the changes.

- Create a Route Table
 - a. Go to **Route Tables** → **Create Route Table**
 - b. Configure:
 - Name: **TestVPC-Public-RT**
 - VPC: **TestVPC**
 - c. Create the route table.
 - d. Select the route table → **Routes** → **Edit Routes**
 - e. Add a new route with:
 - Destination: **0.0.0.0/0**
 - Target: **Internet Gateway (igw-xxx)**
 - f. Save the route.
- Associate the Route Table with the Public Subnet
 - a. Select the route table
 - b. Open **Subnet Associations** → **Edit Subnet Associations**
 - c. Select **TestVPC-Public** subnet
 - d. Save the association.
This makes the subnet a public subnet.
- Launch an EC2 Instance in the Public Subnet
 - a. Go to the EC2 service → **Instances**
 - b. Click **Launch Instance**
 - c. Choose:
 - AML: **Amazon Linux 2**
 - Instance Type: **t2.micro**
 - d. Under **Network Settings**:
 - VPC: **TestVPC**
 - Subnet: **TestVPC-Public**
 - e. Keep storage settings as default.
 - f. Add a tag:
 - Key: **Name**
 - Value: **EC2-A**
 - g. Configure the Security Group:
 - Allow **SSH (Port 22)**
 - Source: **My IP**
 - h. Review and launch the instance.
- Connect to the EC2 Instance
 - a. Use the instance's **Public IP Address**
 - b. Connect from your local machine using:
 - **PuTTY** (Windows) or
 - **Terminal/SSH client** (Mac/Linux)
 - c. Login with the default user: **ec2-user**

VPC Hands-on Part 2

1. Create a Private Subnet

- a. Navigate to **Subnets** → **Create Subnet**
- b. Configure the following details:
 - * Name: **TestVPC-Private**
 - * VPC: **TestVPC**
 - * Availability Zone: **ap-south-1b**
 - * CIDR Block: **10.100.1.0/24**
 - c. Create the subnet.

2. Create a Private Route Table

- a. Go to **Route Tables** → **Create Route Table**
- b. Configure:
 - * Name: **TestVPC-Private-RT**
 - * VPC: **TestVPC**
 - c. Click **Create Route Table**

3. Associate the Private Route Table with the Private Subnet

- a. Select the route table **TestVPC-Private-RT**
- b. Open **Subnet Associations** → **Edit Subnet Associations**
- c. Select **TestVPC-Private** subnet
- d. Save the association.

This makes the subnet a private subnet.

4. Launch another EC2 Instance in the Private Subnet

- a. Launch a new EC2 instance inside the same VPC but select the private subnet:
 - * VPC: **TestVPC**
 - * Subnet: **TestVPC-Private**
 - b. Add the following tag:
 - * Key: **Name**
 - * Value: **EC2-B**

5. Configure a New Security Group for EC2

- a. Create a new security group and add the following inbound rules:
 - * Allow **SSH (Port 22)** from the CIDR range of the Public Subnet*
 - Allow **All ICMP - IPv4** from the CIDR range of the Public Subnet.
- b. Attach this security group to **EC2-B**
- c. Note down the **Private IP Address** of EC2-B.

6. Test Connectivity Between EC2-A and EC2-B

- a. Login to **EC2-A**
- b. Ping the private IP of EC2-B:
 - ping 10.100.1.x

The ping should work successfully.

7. Try SSH Access from EC2-A to EC2-B

- From EC2-A, run:
 - ssh [ec2-user@10.100.1.x](#)
 - Replace `10.100.1.x` with the private IP of EC2-B.
- You will receive a **Permission Denied** error because EC2-A does not yet have the private key required to authenticate.

8. Copy the SSH Key (.pem file) to EC2-A

- a. Open the `.pem` key file on your local machine using a text editor and copy its contents.
- b. On the EC2-A terminal, create a new file:
 - vi key.pem
- c. Press `i` to enter insert mode and paste the key content using right-click or paste shortcut.
- d. Press `Esc` and type:
 - :wq
 - Then press Enter to save and exit.
- e. Change file permissions:
 - chmod 600 key.pem

9. Connect to EC2-B Using the PEM Key

- a. Run the following command from EC2-A:
 - `ssh -i key.pem ec2-user@10.100.1.x`
 - Replace the IP with the private IP address of EC2-B.
- b. SSH access should now work successfully.

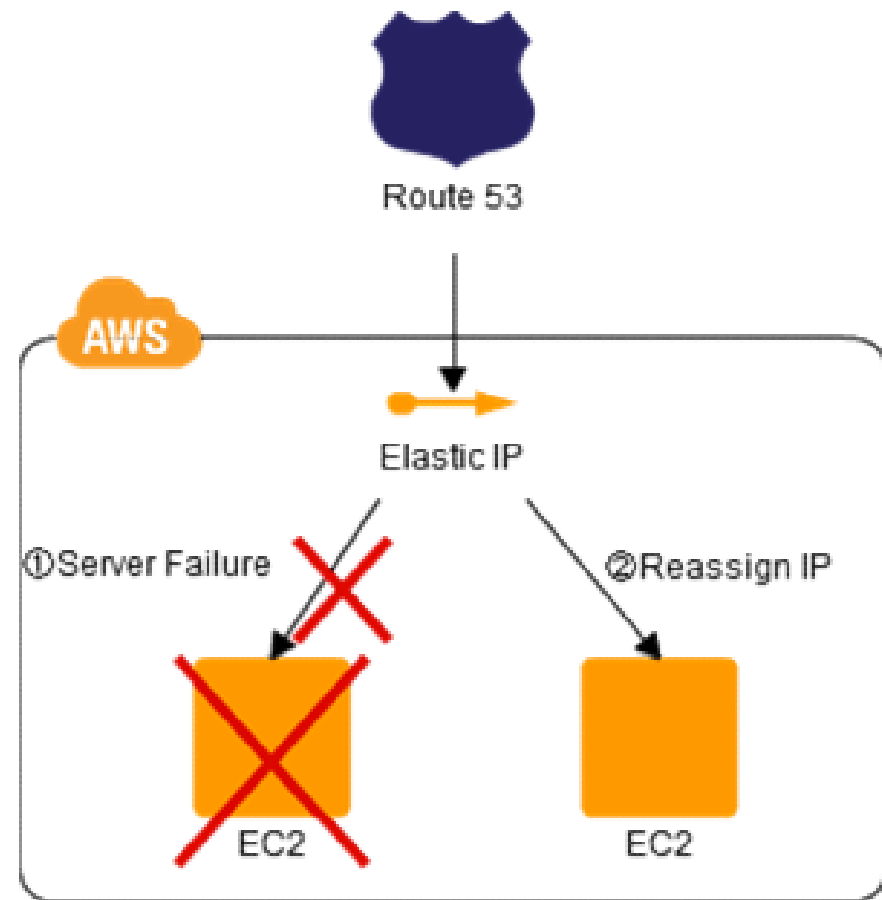
10. Test Internet Access from EC2-B

- a. From EC2-B, run: `ping google.com`
- b. The ping should fail because the private subnet currently does not have internet access.
- c. Since there is no NAT Gateway or Internet Gateway route associated with the private subnet, outbound internet connectivity is not available.

Elastic IP

Elastic IP (EIP)

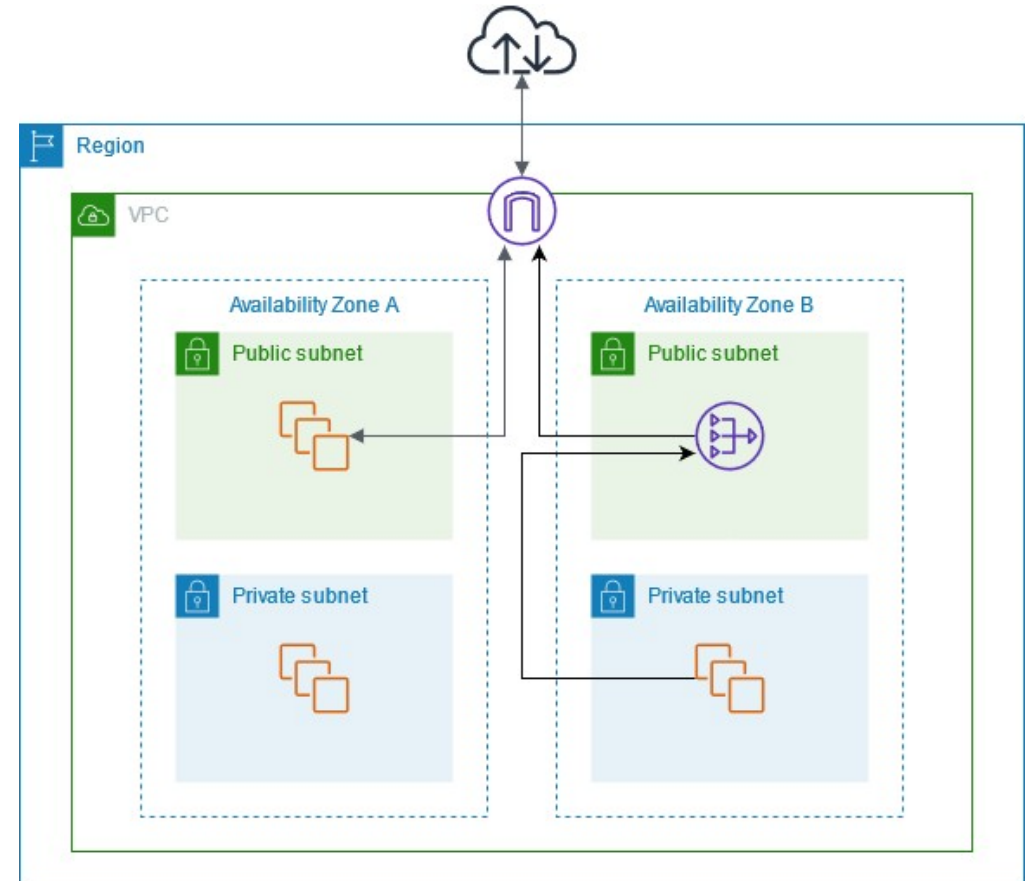
- **Elastic IP (EIP)** is best understood as a **Static, Public IPv4 address reserved for your AWS account**, which you can dynamically remap to different resources.
- Unlike a standard public IP (which is ephemeral), In the cloud, most public IPs are "dynamic." If you stop and start an EC2 instance, it usually gets a brand-new public IP from the AWS pool.
- Technically, an EIP is associated with an **Elastic Network Interface (ENI)**, not the instance itself.
- EIPs are **Region-specific**. You cannot move an EIP from us-east-1 to eu-west-1.
- **Every public IPv4 address costs money.** * **In-use or Idle:** You are charged **\$0.005 per hour** (~\$3.60/month) for every EIP in your account



NAT Gateway

NAT Gateway

- Enables **instances in private subnets** to access Internet.
- NAT Gateway allows outbound internet access without exposing private instances.
- Prevents **incoming connections from the internet**.
- Fully managed by AWS thus no patching or maintenance required
- Setup Requirements VPC with: 1 Public Subnet and 1 Private Subnet
- NAT Gateway Must be created in **Public Subnet**, it **requires** Elastic IP (EIP).



NAT Gateway Routing & Flow

- **In Private Subnet Route Table :**

- 0.0.0.0/0 → NAT Gateway
- All outbound traffic goes via NAT

- **In Public Subnet Route Table:**

- 0.0.0.0/0 → Internet Gateway
- Enables NAT Gateway to reach internet

- **Traffic Flow**

- Private EC2 → NAT Gateway → Internet
- Internet → NAT Gateway → ✗ Blocked

NAT Gateway Insights

- NAT Gateway have 2 interfaces:
 - Outside Interface → **Elastic IP (EIP)** for internet communication
 - Internal Interface → **Private IP (from subnet)** → for internal VPC communication
- NAT GW gets a **private IP from the public subnet CIDR**
 - Example: 10.0.1.x
- NAT GW Uses **Elastic IP (EIP) for** Outside (Internet) interface
 - Example: 3.x.x.x
- As NAT GW is managed by AWS it can automatically scales based on traffic.
- NAT GW (ENI) interface can handle upto ~100 Gbps of Maximum throughput.
- **Hourly Fees:** You pay for the NAT Gateway itself (~\$0.045/hr) plus the hourly cost of the associated Public IPv4.
- **Data Processing:** You are charged per GB of data processed through the gateway.

NAT Gateway Types and Uses

- **Public NAT Gateway:** * **Mandatory EIP:** Requires at least one Elastic IP.
 - **Target:** Used for accessing the public internet.
- **Private NAT Gateway:** * **No EIP:** Uses a private IP from the subnet pool.
 - **Target:** Used for routing between VPCs or to On-Premises networks without internet exposure.

NAT Gateway HA & DR

- **Zonal Nature:** Standard NAT Gateways are Zonal resources. If an Availability Zone (AZ) fails, the NATGW in that zone fails.
- **HA Best Practice:** Deploy **one NAT Gateway per AZ** to ensure zonal independence and avoid inter-AZ data transfer costs.
- **Regional Mode (2026):** A newer managed option that provides a single regional endpoint, eliminating the need for per-AZ management and cross-zone fees.
- **DR Strategy:** NAT Gateways do not auto-failover across regions; you must use Infrastructure as Code (IaC) to redeploy them in a secondary region during a disaster.

NAT Gateway IP Management & Scaling

- **Connection Limits:** A single IP supports ~55,000 concurrent connections to a single destination.
- **IP Pooling:** You can manually associate up to 8 EIPs to one Gateway to prevent **SNAT Port Exhaustion**.
- **Automatic IPAM Pools (2026):** Can be linked to AWS IPAM pools to automatically add or remove IPs based on real-time traffic demand.
- **BYOIP:** You can bring your own public IP ranges to AWS IPAM pools to avoid the hourly "Amazon-owned" IPv4 address charges.

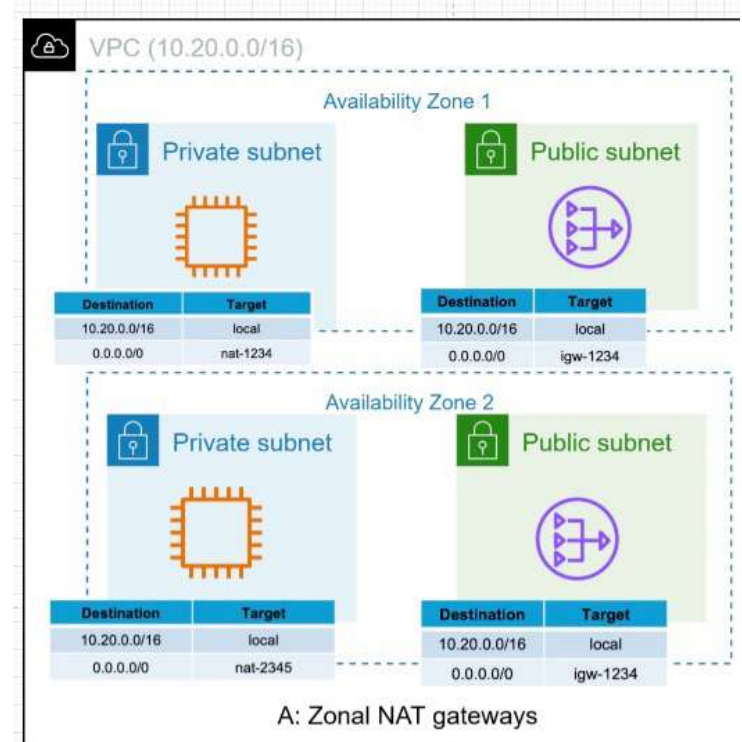
NAT Gateway Design Considerations

- Scenario 1: The "Production Standard" (Multi-AZ)
- Scenario 2: The "Strict Whitelisting" Partnership

Scenario 1

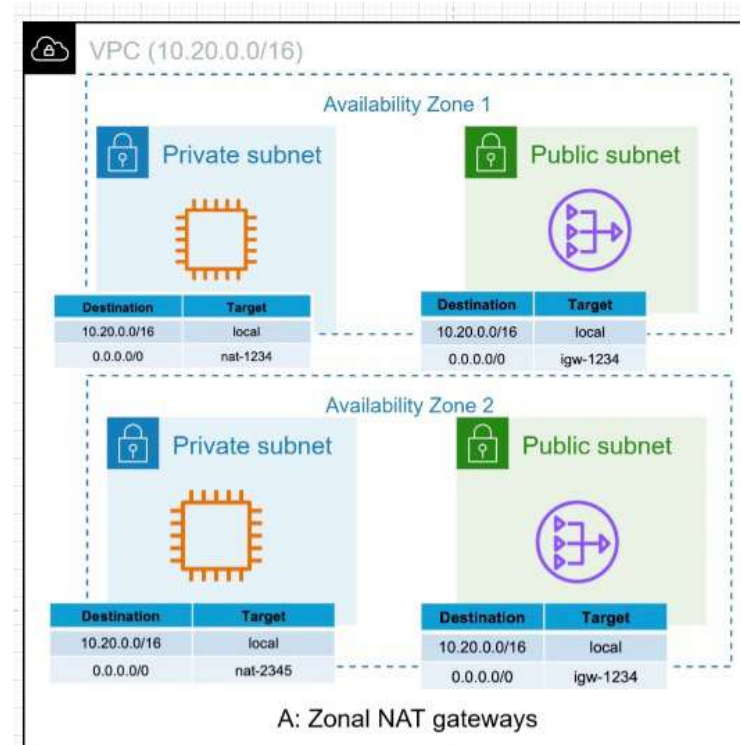
Scenario: You are hosting a high-traffic banking application. Downtime is not an option, and the backend servers in private subnets must always be able to reach external security APIs.

- **Design Strategy:** Deploy **one NAT Gateway per Availability Zone (AZ)**.
- **Reasoning:** NAT Gateways are zonal. If you have only one NAT Gateway and its AZ goes down, your entire application (even parts in other healthy AZs) loses internet connectivity.
- **Routing Logic:**
 - Private Subnet A $\rightarrow$ NAT Gateway A (Public Subnet A)
 - Private Subnet B $\rightarrow$ NAT Gateway B (Public Subnet B)
- **Architect's Tip:** This setup also avoids **Inter-AZ Data Transfer charges** (\$0.01/GB). By keeping traffic local to the AZ, you save significantly on the AWS bill for high-bandwidth apps.



Scenario 2

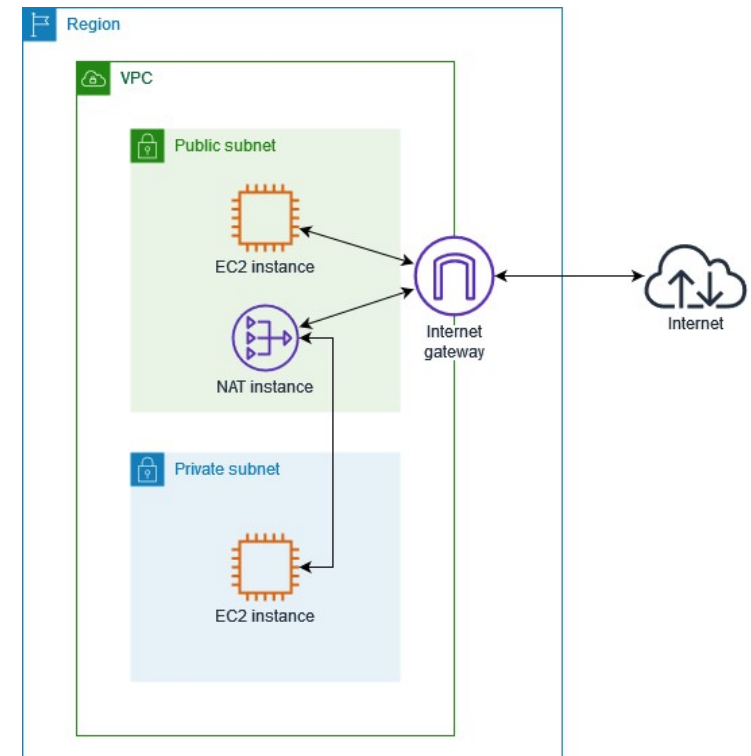
- **Scenario:** You are connecting to a legacy financial institution that requires you to provide a **static IP range** for their firewall. They refuse to whitelist 3-6 individual, non-contiguous Elastic IPs.
 - **The Regional Solution:** Use a Regional NAT Gateway which uses Elastic IP
 - **Why it works:** Since the Regional NATGW have static Elastic IP , you can provide your partner one single IP.



NAT Instance

NAT Instance

- **NAT Instance** is a self-managed EC2 instance configured to perform Network Address Translation.
- Unlike the managed NAT Gateway, you are responsible for the OS, security patching, and scaling of a NAT Instance.
- NAT Instances require an attached Security Group to define exactly which traffic is allowed to pass through from the private subnet to the internet.
- It must reside in a **Public Subnet** and have a Public IP or an Elastic IP assigned to it.
- Bandwidth is strictly tied to the **Instance Type**. If you need more throughput, you must manually stop the instance and change its size (e.g., from a t3.micro to a c5.xlarge).
- It is common to use a NAT Instance as a "Jump Box" or **Bastion Host**



NAT Instance and NAT GW Comparision



Feature	NAT Gateway (Managed)	NAT Instance (Self-Managed)
Management	Zero. AWS handles patching and OS.	High. You patch, monitor, and scale.
Throughput	Scales automatically to 100 Gbps .	Capped by EC2 instance type .
Availability	Built-in redundancy within an AZ.	Single point of failure (unless you script HA).
Security	Managed; supports Network ACLs.	Supports Security Groups & iptables.
Port Forwarding	Not supported.	Supported (via iptables).
Fixed Cost	~\$32.40/month (per AZ).	~\$3.75/month (using t4g.nano).
Usage Cost	\$0.045 per GB processed.	\$0 (No processing fee).

NAT Hand's on

Configure NAT Gateway for Private Subnet Internet Access

Step 1: Create NAT Gateway

Navigate to:
VPC → NAT Gateways → Create NAT Gateway

Configuration:
- Subnet: TestVPC-Public (Must select a Public Subnet)

- Elastic IP: Select "Create New EIP"

Click: Create NAT Gateway

Step 2: Wait for NAT Gateway

- NAT Gateway creation may take 5–10 minutes
- Wait until status becomes: Available / Active

Step 3: Update Private Route Table

Navigate to: VPC → Route Tables

Select: TestVPC-Private Route Table

Routes → Edit Routes → Add Route
Add: Destination: 0.0.0.0/0 , Target: NAT Gateway

Save changes

Step 4: Validate Internet Access

Login to Test-Backend-Host (instance in Private Subnet)

Run: ping google.com

Cheat sheet VPC Basics

1. VPC Basics

- Amazon VPC = logically isolated virtual network in AWS
- Regional service (spans multiple AZs)
- Default VPC exists in every region
- Default VPC includes:
 - Public subnets
 - Internet access enabled

2. CIDR & IP Addressing

- - VPC IPv4 CIDR range:
 - /16 to /28

- - Subnets cannot overlap
- - Supports:
 - - 1 Primary CIDR
 - - Up to 4 Secondary CIDRs
- - AWS reserves 5 IPs per subnet:
 - - Network address
 - - Router
 - - DNS
 - - Future use
 - - Broadcast reservation

Cheat sheet VPC Basics

3. Subnets

- - Subnet is AZ-specific
- - Public subnet:
 - Route to Internet Gateway (IGW)
- - Private subnet:
 - No direct route to IGW
- - Public/Private determined by:
 - Route Table
 - Public IP assignment

4. Internet Gateway (IGW)

- Enables internet connectivity
- Attached to VPC
- Regional and highly available
- Required for public subnet internet access

5. NAT Gateway

- Enables outbound internet for private subnets
- Must be deployed in Public Subnet
- Requires Elastic IP (EIP)
- Managed AWS service
- Supports up to ~100 Gbps
- Route: 0.0.0.0/0 → NAT Gateway

Cheat sheet VPC Basics

6. Security Groups (SG)

- Instance / ENI level firewall
- Stateful
- Allow rules only
- Multiple SGs per ENI supported
- Default:
 - Deny inbound
 - Allow outbound
- Key Feature:
 - Can reference another Security Group

7. Network ACL (NACL)

- Subnet-level firewall
- Stateless
- Supports Allow and Deny
- Rules evaluated in order
- One NACL per subnet
- Traffic Evaluation:
 - Inbound:
 - NACL → Security Group
 - Outbound:
 - Security Group → NACL

Cheat sheet VPC Basics

8. Route Tables

- Controls subnet routing
- One Main Route Table per VPC
- Custom Route Tables supported
- Longest Prefix Match wins
- Common Routes:
 - 0.0.0.0/0 → IGW
 - 0.0.0.0/0 → NAT Gateway
 - VPC CIDR → local

9. High Availability Concepts

- Subnets are AZ-specific
- NAT Gateway is AZ-specific
- Deploy one NAT Gateway per AZ for HA